

```

1 // =====
2 // Builder Pattern - Pizza and Burger
3 // =====
4
5 // %% Product: Pizza %%%%%%%%%%%
6 class Pizza {
7     private String dough;
8     private String sauce;
9     private String topping;
10
11     public void setDough(String dough) {
12         this.dough = dough;
13     }
14
15     public void setSauce(String sauce) {
16         this.sauce = sauce;
17     }
18
19     public void setTopping(String topping) {
20         this.topping = topping;
21     }
22
23     @Override
24     public String toString() {
25         return "Pizza [dough=" + dough + ", sauce=" + sauce + ", topping=" + ...
26     }
27 }
28
29 // %% Product: Burger %%%%%%%%%%%
30 class Burger {
31     private String bun;
32     private String patty;
33     private String salad;
34
35     public void setBun(String bun) {
36         this.bun = bun;
37     }
38
39     public void setPatty(String patty) {
40         this.patty = patty;
41     }
42
43     public void setSalad(String salad) {
44         this.salad = salad;
45     }
46
47     @Override
48     public String toString() {
49         return "Burger [bun=" + bun + ", patty=" + patty + ", salad=" + sala...
50     }
51 }
52
53 // %% Builder Interface: PizzaBuilder %%%%%%%%%%%
54 interface PizzaBuilder {

```

```

55     void reset();
56
57     void setDough(String dough);
58
59     void setSauce(String sauce);
60
61     void setTopping(String topping);
62 }
63
64 // %% Builder Interface: BurgerBuilder %%% %
65 interface BurgerBuilder {
66     void reset();
67
68     void setBun(String bun);
69
70     void setPatty(String patty);
71
72     void setSalad(String salad);
73 }
74
75 // %% Concrete Builder: ConcretePizzaBuilder
76 class ConcretePizzaBuilder implements PizzaBuilder {
77     private Pizza pizza;
78
79     public ConcretePizzaBuilder() {
80         reset();
81     }
82
83     @Override
84     public void reset() {
85         this.pizza = new Pizza();
86     }
87
88     @Override
89     public void setDough(String dough) {
90         pizza.setDough(dough);
91     }
92
93     @Override
94     public void setSauce(String sauce) {
95         pizza.setSauce(sauce);
96     }
97
98     @Override
99     public void setTopping(String topping) {
100         pizza.setTopping(topping);
101     }
102
103     public Pizza getResult() {
104         Pizza result = this.pizza;
105         reset(); // reset builder for next use
106         return result;
107     }
108 }

```

109

```
110 // %% Concrete Builder: ConcreteBurgerBuild
111 class ConcreteBurgerBuilder implements BurgerBuilder {
112     private Burger burger;
113
114     public ConcreteBurgerBuilder() {
115         reset();
116     }
117
118     @Override
119     public void reset() {
120         this.burger = new Burger();
121     }
122
123     @Override
124     public void setBun(String bun) {
125         burger.setBun(bun);
126     }
127
128     @Override
129     public void setPatty(String patty) {
130         burger.setPatty(patty);
131     }
132
133     @Override
134     public void setSalad(String salad) {
135         burger.setSalad(salad);
136     }
137
138     public Burger getResult() {
139         Burger result = this.burger;
140         reset(); // reset builder for next use
141         return result;
142     }
143 }
```

144

```
145 // %% Director: Waiter %%%%%%%%%%
```

```
146 class Waiter {
```

147

```
148     public Pizza constructHawaiianPizza(ConcretePizzaBuilder b) {
149         b.reset();
150         b.setDough("thin crust");
151         b.setSauce("tomato");
152         b.setTopping("ham and pineapple");
153         return b.getResult();
154     }
```

155

```
156     public Burger constructBeefBurger(ConcreteBurgerBuilder b) {
157         b.reset();
158         b.setBun("sesame bun");
159         b.setPatty("beef patty");
160         b.setSalad("lettuce, tomato, onion");
161         return b.getResult();
162     }
```

```

163 }
164
165 // %% Client / Main %%%%%%%%%%%%%%%
166 public class BuilderPattern {
167     public static void main(String[] args) {
168
169         Waiter waiter = new Waiter();
170
171         // Build a Hawaiian Pizza
172         ConcretePizzaBuilder pizzaBuilder = new ConcretePizzaBuilder();
173         Pizza hawaiianPizza = waiter.constructHawaiianPizza(pizzaBuilder);
174         System.out.println("Ordered: " + hawaiianPizza);
175
176         // Build a Beef Burger
177         ConcreteBurgerBuilder burgerBuilder = new ConcreteBurgerBuilder();
178         Burger beefBurger = waiter.constructBeefBurger(burgerBuilder);
179         System.out.println("Ordered: " + beefBurger);
180
181         // Custom order using builder directly (without director)
182         System.out.println("\n--- Custom Order (no director) ---");
183         pizzaBuilder.reset();
184         pizzaBuilder.setDough("thick crust");
185         pizzaBuilder.setSauce("bbq");
186         pizzaBuilder.setTopping("pepperoni");
187         Pizza customPizza = pizzaBuilder.getResult();
188         System.out.println("Custom: " + customPizza);
189     }
190 }

```